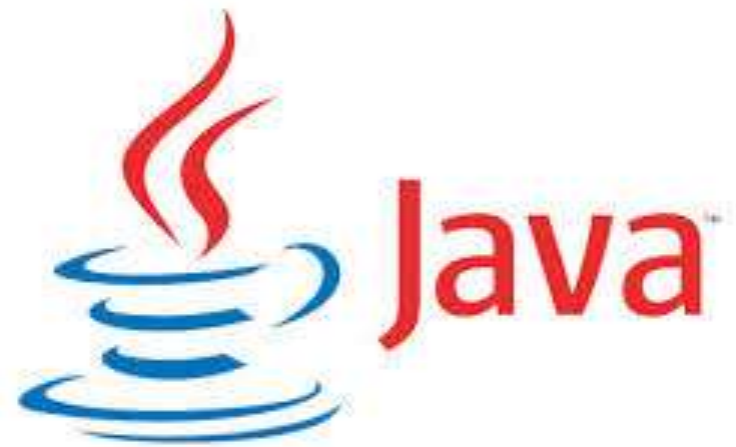




SOC2030

Application Programming in Java



Week (4) 5

Dr. Andrei Dragunov

- Abstract Class
- Abstract class inheritance
- Purpose of Interfaces
- Interface Declaration
- Implementing an Interface
- Interface Inheritance

https://www.tutorialspoint.com/compile_java_online.php

Key Point

- *A superclass defines common behavior for related subclasses.*
- *An interface can be used to define common behavior for classes (including unrelated classes).*

Abstract Classes

- *An abstract class cannot be used to create objects. An abstract class can contain abstract methods, which are implemented in concrete subclasses.*
- In the inheritance hierarchy, classes become more specific and concrete *with each new subclass*. If you move from a subclass back up to a superclass, the classes become more general and less specific.
- Sometimes a superclass is so abstract that it cannot be used to create any specific instances. Such a class is referred to as an *abstract class*.

Declaring an Abstract Class and Abstract Methods

You make a class abstract by declaring it with keyword **abstract**. An abstract class normally contains one or more **abstract methods**. An abstract method is an *instance method* with keyword **abstract** in its declaration, as in

```
public abstract void draw(); // abstract method
```

Abstract methods do *not* provide implementations.

A class that contains *any* abstract methods must be explicitly declared abstract even if that class contains some concrete (non abstract) methods.

Each concrete subclass of an abstract superclass also must provide concrete implementations of each of the superclass's abstract methods.

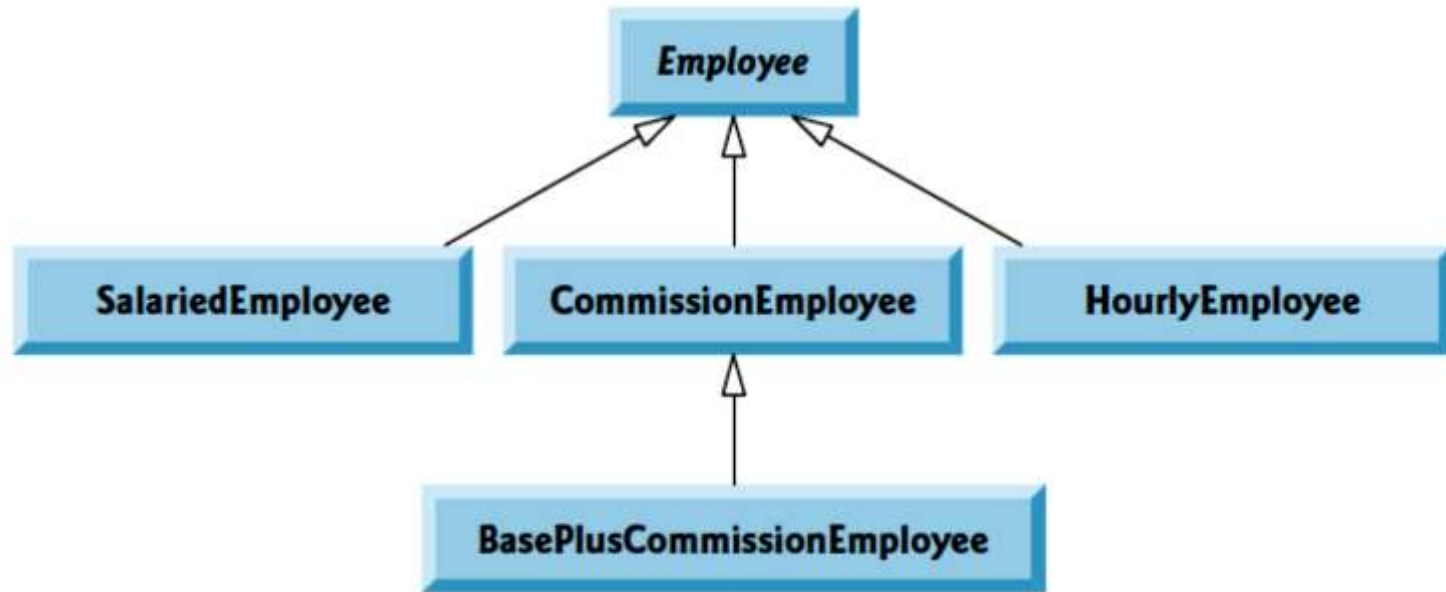
Constructors and static methods cannot be declared abstract.

- Constructors are *not* inherited, so an abstract constructor could never be implemented.
- Though non-private static methods are inherited, they cannot be overridden. Since abstract methods are meant to be overridden so that they can process objects based on their types, it would not make sense to declare a static method as abstract.

Using Abstract Classes to Declare Variables

- We cannot instantiate objects of abstract superclasses, but we *can* use abstract superclasses to declare variables that can hold references to objects of *any* concrete class *derived from* those abstract superclasses.
- We'll use such variables to manipulate subclass objects *polymorphically*. You also can use abstract superclass names to invoke static methods declared in those abstract superclasses.

Employee hierarchy UML class diagram.



- Abstract superclass **Employee** declares the “interface” to the hierarchy—that is, the set of methods that a program can invoke on all **Employee** objects. We use the term “interface” here in a *general* sense to refer to the various ways programs can communicate with objects of *any* **Employee** subclass. Be careful not to confuse the general notion of an “interface” with the formal notion of a Java interface!

	earnings	toString
Employee	abstract	<i>firstName lastName</i> social security number: <i>SSN</i>
Salaried- Employee	weeklySalary	salaried employee: <i>firstName lastName</i> social security number: <i>SSN</i> weekly salary: <i>weeklySalary</i>
Hourly- Employee	<pre> if (hours <= 40) wage * hours else if (hours > 40) { 40 * wage + (hours - 40) * wage * 1.5 } </pre>	hourly employee: <i>firstName lastName</i> social security number: <i>SSN</i> hourly wage: <i>wage</i> ; hours worked: <i>hours</i>
Commission- Employee	commissionRate * grossSales	commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i>
BasePlus- Commission- Employee	(commissionRate * grossSales) + baseSalary	base salaried commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i> ; base salary: <i>baseSalary</i>

Abstract Superclass Employee

Class Employee provides methods `earnings` and `toString()`, in addition to the *get* methods that return the values of Employee's instance variables. An `earnings` method certainly applies *generically* to all employees. But each earnings calculation depends on the employee's particular class. So we declare `earnings` as abstract in superclass Employee because a *specific* default implementation does not make sense for that method—there isn't enough information to determine what amount `earnings` should return.

Each subclass overrides `earnings` with an appropriate implementation.

```
public abstract class Employee {
    private final String firstName;
    private final String lastName;
    private final String socialSecurityNumber;

    // constructor
    public Employee(String firstName, String lastName, String socialSecurityNumber)
    {
        this.firstName = firstName;
        this.lastName = lastName;
        this.socialSecurityNumber = socialSecurityNumber;
    }

    // return String representation of Employee object
    @Override
    public String toString()
    {
        return String.format("%s %s\nsocial security number: %s",
            getFirstName(), getLastName(), getSocialSecurityNumber());
    }

    // abstract method must be overridden by concrete subclasses
    public abstract double earnings();
}
```

Built-in Annotations in Java @Override:

- While overriding a method in the child class, we should use this annotation to mark that method. This makes code readable and avoid maintenance issues, such as: while changing the method signature of parent class, you must change the signature in child classes (where this annotation is being used) otherwise compiler would throw compilation error. This is difficult to trace when you haven't used this annotation.

Concrete Subclass SalariedEmployee

Class SalariedEmployee extends class Employee and overrides *abstract* method earnings, which makes SalariedEmployee a *concrete* class.

```
public class SalariedEmployee extends Employee{
    private double weeklySalary;

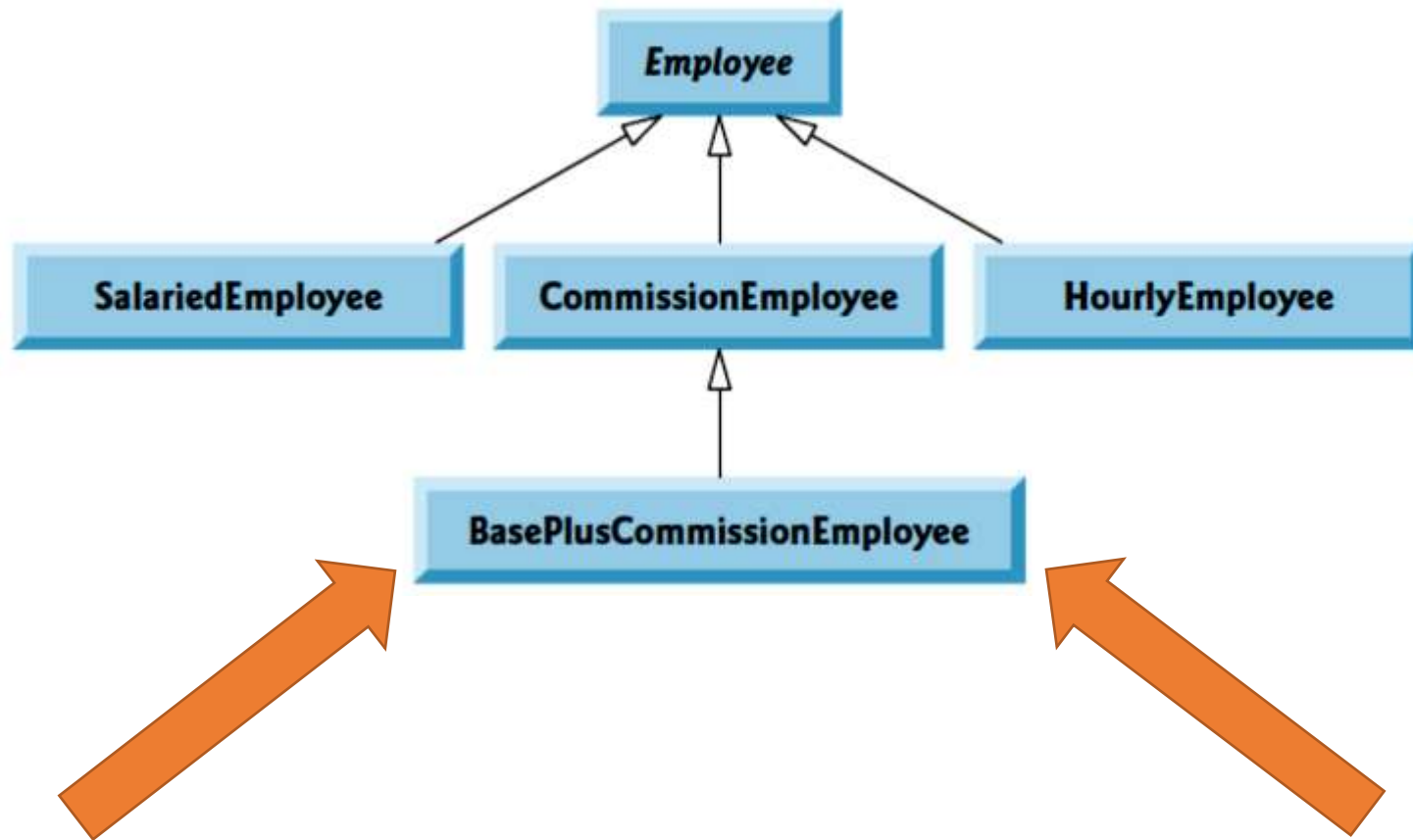
    // constructor
    public SalariedEmployee(String firstName, String lastName,
        String socialSecurityNumber, double weeklySalary)
    {
        super(firstName, lastName, socialSecurityNumber);
        this.weeklySalary = weeklySalary;
    }
}
```



```
// calculate earnings; override abstract method earnings in Employee
@Override
public double earnings()
{
    return getWeeklySalary();
}

// return String representation of SalariedEmployee object
@Override
public String toString()
{
    return String.format("salaried employee: %s\n%s: $%,.2f",
        super.toString(), "weekly salary", getWeeklySalary());
}
```

Indirect Concrete Subclass BasePlusCommissionEmployee



Class **BasePlusCommissionEmployee** extends class **CommissionEmployee** and therefore is an *indirect* subclass of class **Employee**.

Polymorphic Processing, Operator instance of and Downcasting

```
Employee[] employees = new Employee[2];
// initialize array with Employees
employees[0] = salariedEmployee;
employees[1] = hourlyEmployee;
System.out.println("Employees processed polymorphically:");

// generate report
for (Employee e : employees) {
    if (e instanceof SalariedEmployee) {
        SalariedEmployee salariedEmployee = (SalariedEmployee) e;
        System.out.println("Salaried employee: John Smith");
        System.out.println("social security number: 111-11-1111");
        System.out.println("weekly salary: $800.00");
    } else if (e instanceof HourlyEmployee) {
        HourlyEmployee hourlyEmployee = (HourlyEmployee) e;
        System.out.println("hourly employee: Karen Price");
        System.out.println("social security number: 222-22-2222");
        System.out.println("hourly wage: $16.75; hours worked: 40.00");
    }
}

BUILD SUCCESSFUL (total time: 0 seconds)
```

A Deeper Explanation of Issues with Calling Methods from Constructors

- **Do not call overridable methods from constructors.**
- When creating a *subclass* object, this could lead to an overridden method being called before the *subclass* object is fully initialized.
- Recall that when you construct a *subclass* object, its constructor first calls one of the direct *superclass*'s constructors. If the *superclass* constructor calls an overridable method, the *subclass*'s version of that method will be called by the *superclass* constructor—before the *subclass* constructor's body has a chance to execute.

```
class A{
    public int a;
    public A() {
        mA();
    }

    public void mA() {
        System.out.println("Method in A");
        a += 5;
    }
}
```

```
class B extends A{
    public B() {
        mA();
        a += 10;
    }

    public void mA() {
        System.out.println("Method in B");
        a += 1;
    }
}
```

run:

Method in B

Method in B

12

BUILD SUCCESSFUL

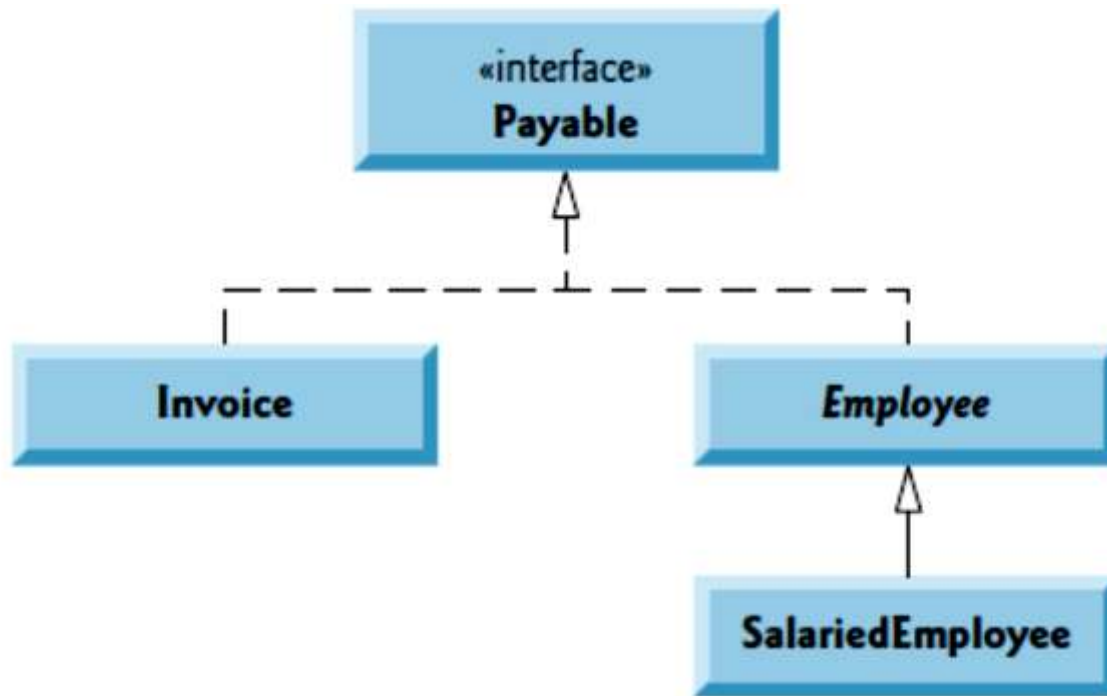
Standardizing Interactions

Interfaces define and standardize the ways in which things such as people and systems can interact with one another. For example, the controls on a radio serve as an interface between radio users and a radio's internal components. The controls allow users to perform only a limited set of operations (e.g., change the station, adjust the volume, choose between AM and FM), and different radios may implement the controls in different ways (e.g., using push buttons, dials, voice commands). The interface specifies *what* operations a radio must permit users to perform but does not specify *how* the operations are performed.

Software Objects Communicate Via Interfaces

Software objects also communicate via interfaces. A Java interface describes a set of methods that can be called on an object to tell it, for example, to perform some task or return some piece of information. The next example introduces an interface named Payable to describe the functionality of any object that must be “capable of being paid” and thus must offer a method to determine the proper payment amount due. An **interface declaration** begins with the keyword **interface** and contains *only* constants and abstract methods.

Unlike classes, all interface members *must* be public, and *interfaces may not specify any implementation details*, such as concrete method declarations and instance variables. All methods declared in an interface are implicitly public abstract methods, and all fields are implicitly public, static and final.



```
package week5_example;
```

```
public interface Payable {
```

```
    double getPaymentAmount(); // calculate payment; no implementation  
}
```

Using an Interface

- To use an interface, a concrete class must specify that it **implements** the interface and must declare each method in the interface with the signature specified in the interface declaration.
- To specify that a class implements an interface, add the `implements` keyword and the name of the interface to the end of your class declaration's first line.
- A class that does not implement *all* the methods of the interface is an *abstract* class and must be declared `abstract`. Implementing an interface is like signing a *contract* with the compiler that states, **“I will declare all the methods specified by the interface or I will declare my class abstract.”**

```
public abstract class Employee implements Payable{
```


Interfaces vs. Abstract Classes

An interface is often used in place of an abstract class when there's no default implementation to inherit—that is, no fields and no default method implementations.

Like public abstract classes, interfaces are typically public types. Like a public class, a public interface must be declared in a file with the same name as the interface and the .java filename extension.

```
public class testPayable implements Payable {  
  
    @Override  
    public double getPaymentAmount() {  
        throw new UnsupportedOperationException("Not supported yet.");  
//To change body of generated methods, choose Tools | Templates.  
    }  
  
}
```

Note!

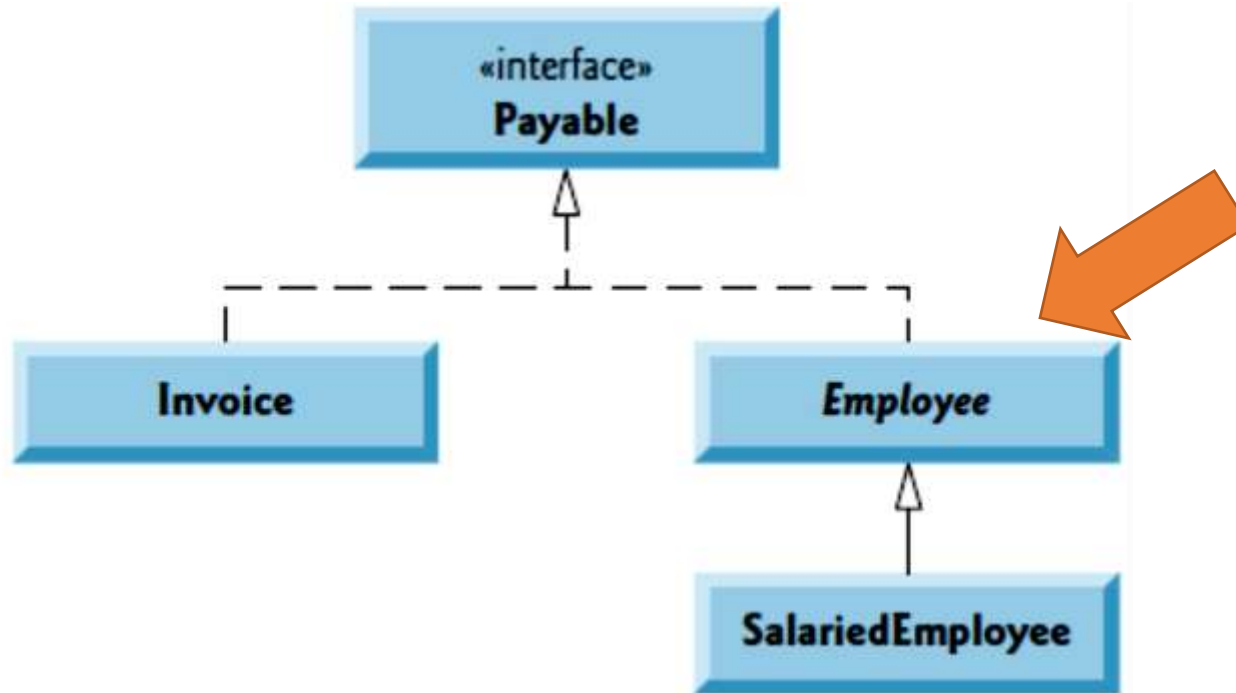
- ***A Class Can Extend Only One Other Class But Can Implement Many Interfaces***

```
interface A{
    public void mA();
}
interface B{
    public void mB();
}
class C implements A, B{

    public void mA() {
        System.out.println("Not yet");
    }

    public void mB() {
        System.out.println("Not yet");
    }
}
```

Back to our Payable hierarchy



Abstract Class

- SalariedEmployee class that implements interface Payable method getPaymentAmount().

```
// calculate earnings; implement interface Payable method that was  
// abstract in superclass Employee  
@Override  
public double getPaymentAmount()  
{  
    return getWeeklySalary();  
}
```

Invoice class that implements Payable.

- P-pp. 424-426 (Main book)

```
// Fig. 10.12: Invoice.java
```

```
// Invoice class that implements Payable.
```

```
public class Invoice implements Payable
```

```
{
```

```
    private final String partNumber;
```

```
    private final String partDescription;
```

```
    private int quantity;
```

```
    private double pricePerItem;
```

```
    // constructor
```

```
    public Invoice(String partNumber, String partDescription, int quantity,  
                  double pricePerItem)
```

```
    {
```

Using Interface Payable to Process Invoices and Employees Polymorphically

```
public static void main(String[] args)
{
    // create four-element Payable array
    Payable[] payableObjects = new Payable[4];

    // populate array with objects that implement Payable
    payableObjects[0] = new Invoice("01234", "seat", 2, 375.00);
    payableObjects[1] = new Invoice("56789", "tire", 4, 79.95);
    payableObjects[2] =
        new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
    payableObjects[3] =
        new SalariedEmployee("Lisa", "Barnes", "888-88-8888", 1200.00);

    System.out.println(
        "Invoices and Employees processed polymorphically:");

    // generically process each element in array payableObjects
    for (Payable currentPayable : payableObjects)
    {
```



```
// output currentPayable and its appropriate payment amount
System.out.printf("%n%s %n%s: $%,.2f%n",
    currentPayable.toString(), // could invoke implicitly
    "payment due", currentPayable.getPaymentAmount());
}
} // end main
} // end class PayableInterfaceTest
```


Invoices and Employees processed polymorphically:

invoice:

part number: 01234 (seat)

quantity: 2

price per item: \$375.00

payment due: \$750.00

invoice:

part number: 56789 (tire)

quantity: 4

price per item: \$79.95

payment due: \$319.80

salaried employee: John Smith

social security number: 111-11-1111

weekly salary: \$800.00

payment due: \$800.00

salaried employee: Lisa Barnes

social security number: 888-88-8888

weekly salary: \$1,200.00

payment due: \$1,200.00

default Interface Methods

```
interface A{
    public default void mA() {
        System.out.println("Default mA");
    };
}

interface B{
    public void mB();
}

class C implements A, B{
    public void mB() {
        System.out.println("Not yet");
    }
}
```

```
interface A{  
    public default void mA() {  
        System.out.println("Default mA");  
    };  
}  
interface B{  
    public void mB();  
    public void mA();  
}  
class C implements A, B{
```


Interface	Description
<code>Serializable</code>	An interface used to identify classes whose objects can be written to (i.e., serialized) or read from (i.e., deserialized) some type of storage (e.g., file on disk, database field) or transmitted across a network. We use <code>Serializable</code> in Chapter 15, Files, Streams and Object Serialization, and Chapter 28, Networking.
<code>Runnable</code>	Implemented by any class that represents a task to perform. Objects of such as class are often executed in parallel using a technique called <i>multithreading</i> (discussed in Chapter 23, Concurrency). The interface contains one method, <code>run</code> , which specifies the behavior of an object when executed.
GUI event-listener interfaces	You work with graphical user interfaces (GUIs) every day. In your web browser, you might type the address of a website to visit, or you might click a button to return to a previous site. The browser responds to your interaction and performs the desired task. Your interaction is known as an <i>event</i> , and the code that the browser uses to respond to an event is known as an <i>event handler</i> . In Chapter 12, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2, you'll learn how to build GUIs and event handlers that respond to user interactions. Event handlers are declared in classes that implement an appropriate <i>event-listener interface</i> . Each event-listener interface specifies one or more methods that must be implemented to respond to user interactions.
<code>AutoCloseable</code>	Implemented by classes that can be used with the <code>try-with-resources</code> statement (Chapter 11, Exception Handling: A Deeper Look) to help prevent resource leaks.

Interface	Description
Comparable	Java contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare primitive values. However, these operators <i>cannot</i> be used to compare objects. Interface Comparable is used to allow objects of a class that implements the interface to be compared to one another. Interface Comparable is commonly used for ordering objects in a collection such as an array. We use Comparable in Chapter 16, Generic Collections, and Chapter 20, Generic Classes and Methods.